# Memoization

zation · Cache coin-change DP / tax-rate lookups with call-count trac · Build with

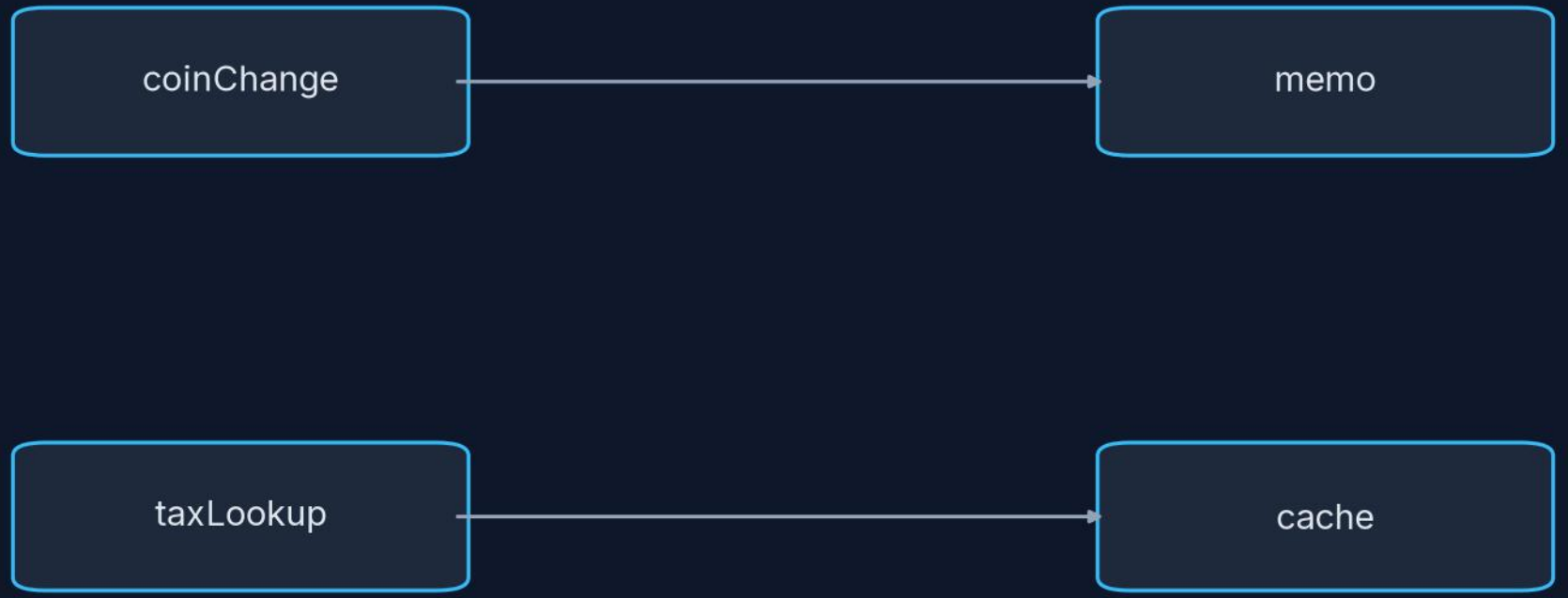
# Today's Hook

- Problem: Cache coin-change DP / tax-rate lookups with call-count trace
- Pattern: Memoization — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Cache coin-change DP / tax-rate lookups with call-count trace
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Memoization: Cache coin-change DP / tax-rate lookups with call-

# Real-World Scenario

- Running example: Cache coin-change DP / tax-rate lookups with call-count trace
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.functional.memoization.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Memoization: coin-change DP + tax lookup ===");
        int[] coins = {1, 5, 10, 25};
        int amount = 30;

        var naiveC = new CoinChangeMemo.Counter();
        long t0 = System.nanoTime();
        int naive = CoinChangeMemo.naiveWays(amount, coins, naiveC);
        long naiveMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("naive ways=" + naive + " calls=" + naiveC.get() + " ms=" + naiveMs);

        var memoC = new CoinChangeMemo.Counter();
        t0 = System.nanoTime();
        int memo = CoinChangeMemo.memoWays(amount, coins, memoC);
        long memoMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("memo  ways=" + memo + " calls=" + memoC.get() + " ms=" + memoMs);

        System.out.println("--- tax-rate lookup memoize ---");
        var taxC = new CoinChangeMemo.Counter();
        var tax = CoinChangeMemo.memoize(CoinChangeMemo::expensiveTaxRate, taxC);
        long t1 = System.nanoTime();
        System.out.println("CA1=" + tax.apply("CA") + " CA2=" + tax.apply("CA") + " NY=" + tax.apply("NY"));
        System.out.println("computeCalls=" + taxC.get() + " elapsedMs=" + (System.nanoTime() - t1) / 1_000_000);
    }
```

# Naive Code (2)

```
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```java
package com.buildwithwaqar.patterns.functional.memoization.problem;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Function;

/** Memoized coin-change DP + tax-rate lookup with call-count / timing traces. */
public final class CoinChangeMemo {
    private CoinChangeMemo() {}

    public static final class Counter {
        private final AtomicInteger calls = new AtomicInteger();
        public int inc() { return calls.incrementAndGet(); }
        public int get() { return calls.get(); }
        public void reset() { calls.set(0); }
    }

    /** Naive exponential recursion. */
    public static int naiveWays(int amount, int[] coins, Counter c) {
        c.inc();
        if (amount == 0) return 1;
        if (amount < 0 || coins.length == 0) return 0;
        int with = naiveWays(amount - coins[0], coins, c);
        int without = naiveWays(amount, Arrays.copyOfRange(coins, 1, coins.length), c);
        return with + without;
    }
```

# Solution (2)

```java
public static int memoWays(int amount, int[] coins, Counter c) {
    return memoWays(amount, coins, 0, new HashMap<>(), c);
}

private static int memoWays(int amount, int[] coins, int i, Map<String, Integer> cache, Counter c) {
    c.inc();
    if (amount == 0) return 1;
    if (amount < 0 || i >= coins.length) return 0;
    String key = amount + ":" + i;
    Integer hit = cache.get(key);
    if (hit != null) return hit;
    int with = memoWays(amount - coins[i], coins, i, cache, c);
    int without = memoWays(amount, coins, i + 1, cache, c);
    int ans = with + without;
    cache.put(key, ans);
    return ans;
}

/** Generic memoize wrapper for expensive lookups. */
public static <A, B> Function<A, B> memoize(Function<A, B> f, Counter c) {
    Map<A, B> cache = new HashMap<>();
    return a -> {
        if (cache.containsKey(a)) return cache.get(a);
        c.inc();
        B b = f.apply(a);
        cache.put(a, b);
        return b;
    };
}
```

# Solution (3)

```
    public static int expensiveTaxRate(String region) {
        try { Thread.sleep(5); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        return switch (region) {
            case "CA" -> 725; case "NY" -> 800; default -> 500;
        };
    }
}
```

# Runnable main

```java
package com.buildwithwaqar.patterns.functional.memoization.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Memoization: coin-change DP + tax lookup ===");
        int[] coins = {1, 5, 10, 25};
        int amount = 30;

        var naiveC = new CoinChangeMemo.Counter();
        long t0 = System.nanoTime();
        int naive = CoinChangeMemo.naiveWays(amount, coins, naiveC);
        long naiveMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("naive ways=" + naive + " calls=" + naiveC.get() + " ms=" + naiveMs);

        var memoC = new CoinChangeMemo.Counter();
        t0 = System.nanoTime();
        int memo = CoinChangeMemo.memoWays(amount, coins, memoC);
        long memoMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("memo  ways=" + memo + " calls=" + memoC.get() + " ms=" + memoMs);

        System.out.println("--- tax-rate lookup memoize ---");
        var taxC = new CoinChangeMemo.Counter();
        var tax = CoinChangeMemo.memoize(CoinChangeMemo::expensiveTaxRate, taxC);
        long t1 = System.nanoTime();
        System.out.println("CA1=" + tax.apply("CA") + " CA2=" + tax.apply("CA") + " NY=" + tax.apply("NY"));
        System.out.println("computeCalls=" + taxC.get() + " elapsedMs=" + (System.nanoTime() - t1) / 1_000_000);
    }
```

# Runnable main (2)

```
}
```

# Walkthrough Output

```
=== Memoization: coin-change DP + tax lookup ===
naive ways=18 calls=961 ms=0
memo  ways=18 calls=241 ms=1
--- tax-rate lookup memoize ---
CA1=725 CA2=725 NY=800
computeCalls=2 elapsedMs=12
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Memoization.

# Related Patterns

- Contrast Memoization with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Memoization → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.